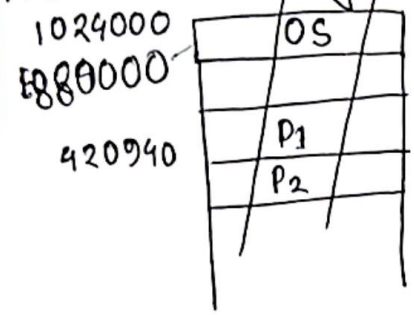
Semaphores, Next Sunday
Dining Philosophers, Quiz - 3
Post midterm -
Deadlock,
Synchronization,

Main memory

- Main memory consists of RAM & ROM
- Main memory (RAM) & Registers are the only storages that cpu directly accesses

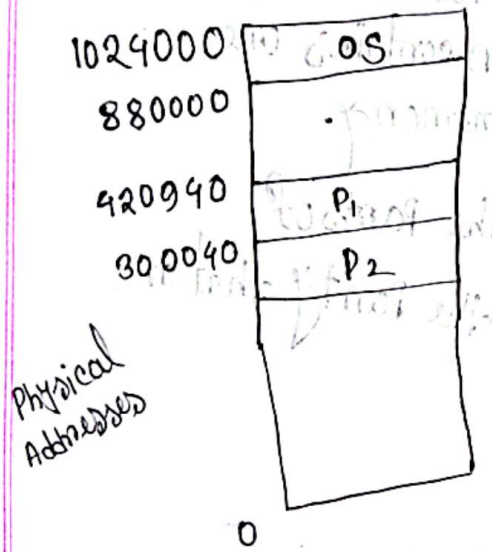
ROM → Startup

Processes occupy the RAM

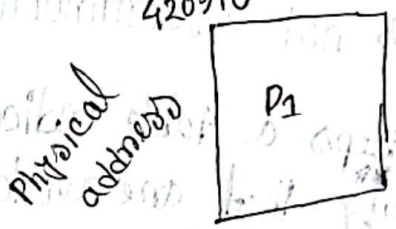


base register

Processes occupy the RAM

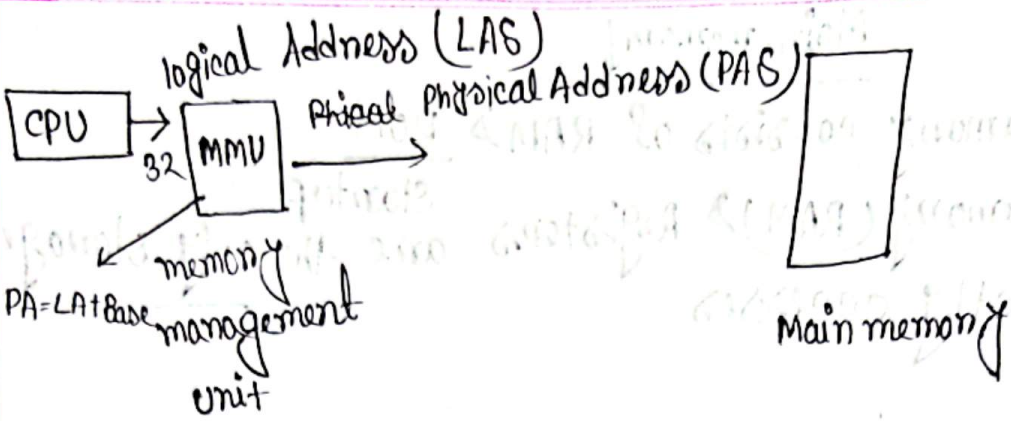


limit register 420940



base register 300040

#Every process thinks their own address starts from 0 (the very first spot)



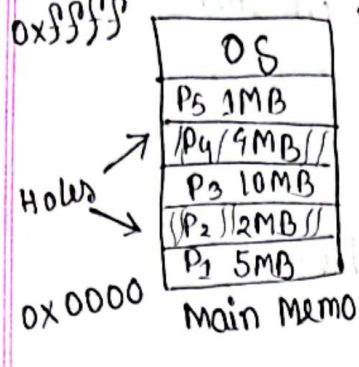
if LA+base > limit
throw error

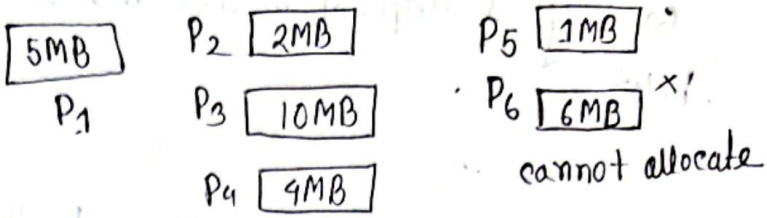
Main memory Allocation

Dynamic partitioning for contiguous memory Allocation

- Assign processes to variable sized partitions in memory, where each partition contains one process & not discontinuous in memory.
- The OS keeps a table indicating the parts of main memory that are available & the parts that are occupied.

External Fragmentation (cause of holes)
↓ why - that's bad





- Initially all memory is available as a large block.
→ When processes are done executing, they leave the main memory P2 & P4 finishes

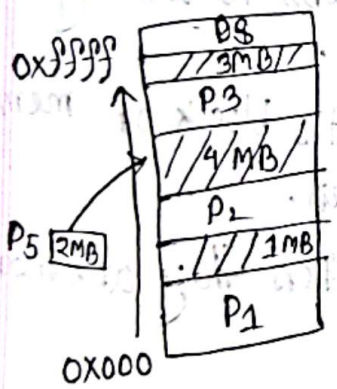
→ When a process arrives & needs memory, the system searches the set of holes that is large enough for the process
- If no holes are avilable, send the process to waiting queue or reject the process entirely.
- When a process terminates it releases it's block of memory which is then added to the set of holes.
→ if 2 holes are adjacent to one another they are merged.

→ this Procedure is a particular instance of a dynamic storage allocation Problem, which tries to satisfy a request of size n from a list of free holes.

→ There are many algorithms to this Problem, the most commonly used were:

① First Fit
② Best Fit
③ Worst Fit

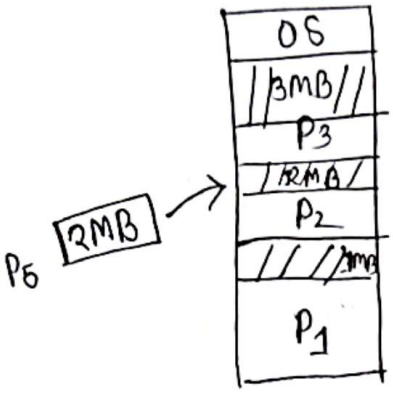
First Fit: Allocates the first holes that is large enough linear search

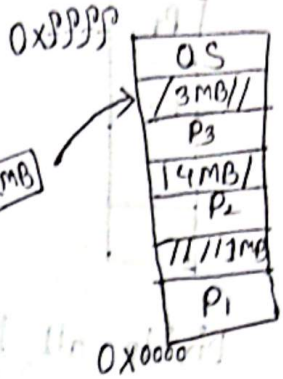


time complexity O(n) → linear

Best Fit: Allocates the smallest hole that is big enough
→ Must scan the entire list of available holes
→ this strategy produces the smallest leftover holes.

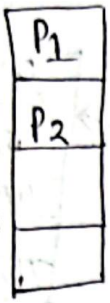
Worst Fit: Allocates the largest hole that can accomodate the process.
- May be useful for future large requests.



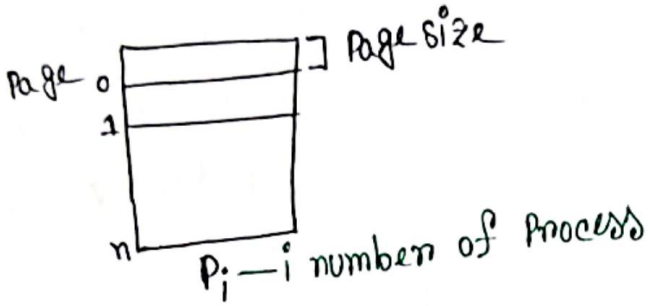


21-07-26

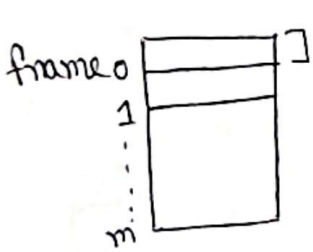
Non-contiguous Memory Allocation:



Divide all the Processes into Pages.

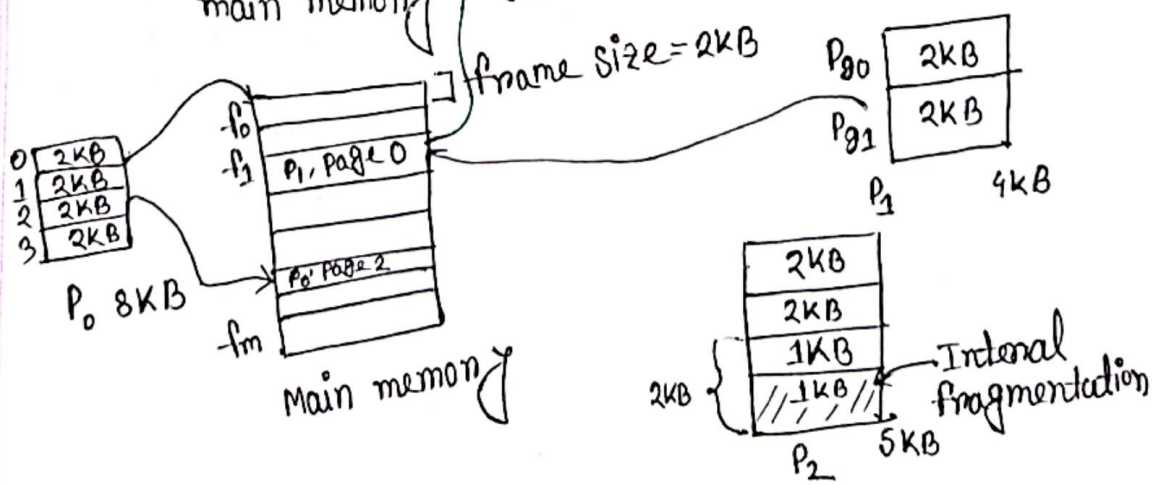


Divide the main memory into Frames

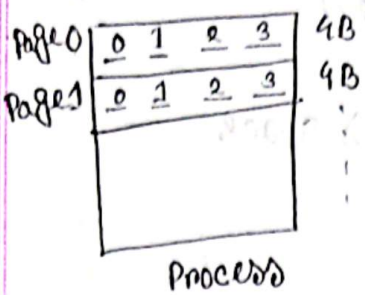


Page size = Frame Size

When Process execution done, when a page leaves, it will always leave behind the same hole size.

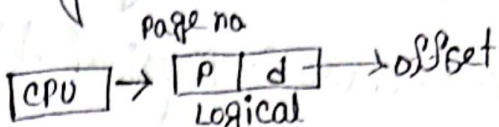


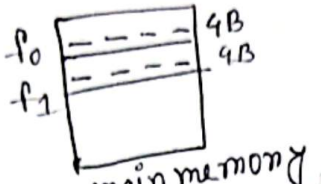
Pages

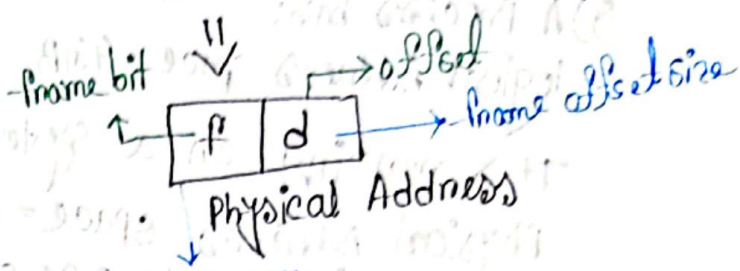


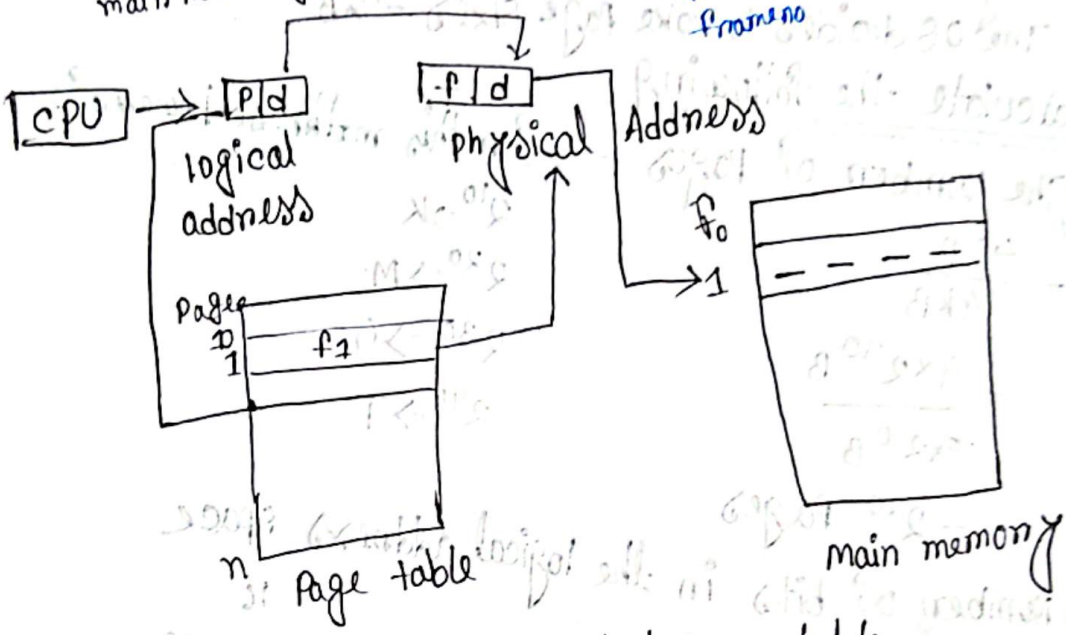
Memory is Byte (B) Addressable

1 byte = 8 bits









- Every process has a dedicated Page table
- Every Page & it's corresponding frame no are saved in the Page table
- The Physical Address consists of 2 parts:
  1) The frame number, which tells us where that particular Page is in main memory

2. The offset (d), which contains the location of the particular byte that is being referenced.

Logical Address space & Physical Address space

Q) A process has:
logical Address space = 4GB
It's running on a system with:
Physical Address space = 64 MB
The OS decides to make page sizes = 4KB

calculate the following → for this mathuse Power of 2

$2^{10} \Rightarrow K$
$2^{20} \Rightarrow M$
$2^{30} \Rightarrow G$
$2^{40} \Rightarrow T$

a) The Number of pages

$$= \frac{4GB}{4KB}$$

$$= \frac{4 \times 2^{30} B}{4 \times 2^{10} B}$$

$$= 2^{20} \text{ pages}$$

b) Number of bits in the logical address space

$$= 4GB$$
$$= 4 \times 2^{30} B$$
$$= 2^2 \times 2^{30} B$$
$$= 2^{32} B$$

I have need 32 bits

16
0 --- 15
1111 2^4 ...
4

→ 20 bits — 12 bits →

p	d

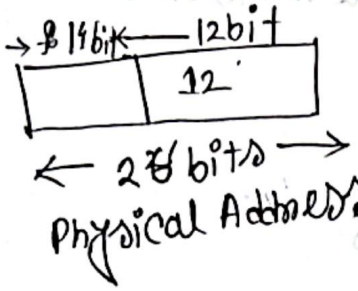
Logical address

← 32 bits →

c) Size of each Page / No of divisons in each Page

$\rightarrow 2^{12}$

d) NO of frames



$64\ MB$

$= 2^6 \times 2^{20} B$

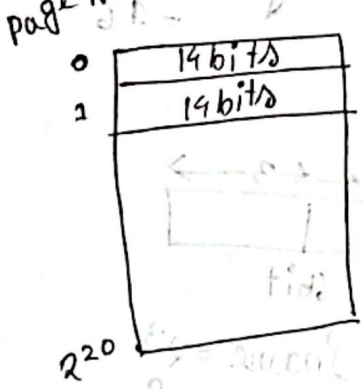
$= 2^{26} B$

$= 2^{14}$

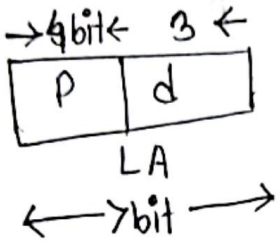
e) Size of Page table

$= \frac{14 \times 2^{20}}{8}$

$= 1.75\ MB$



9) consider a system which has logical Address = 7 bits
Physical Address = 6 bits
Page size = 8 B
calculate the number of Pages, the number of Frames size of the table



8 B
2^3 B

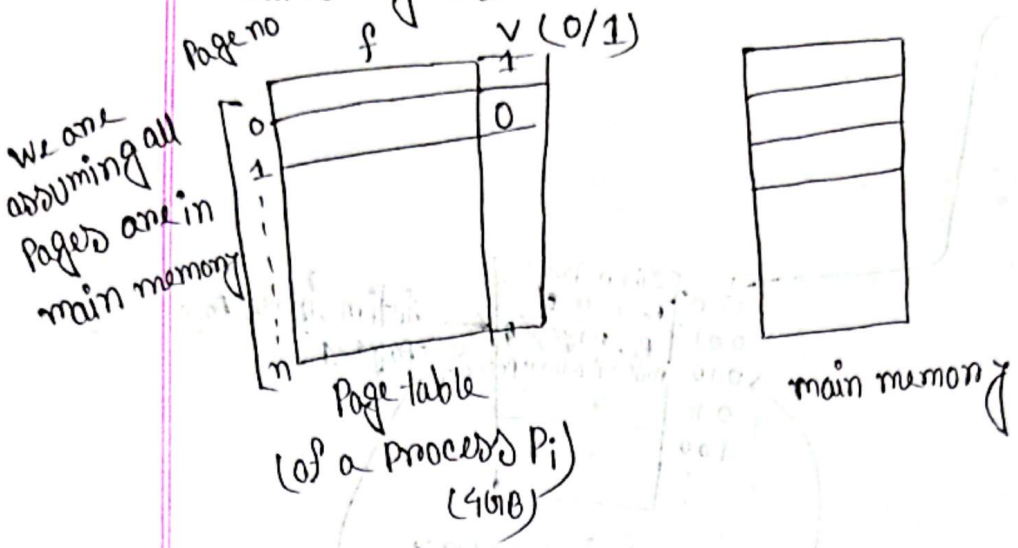
No of Pages = 2^4
= 16

No of frame = 2^3
= 8

size of page table = $\frac{16 \times 3}{8}$
= 6 B

25-4-26

What is Page table



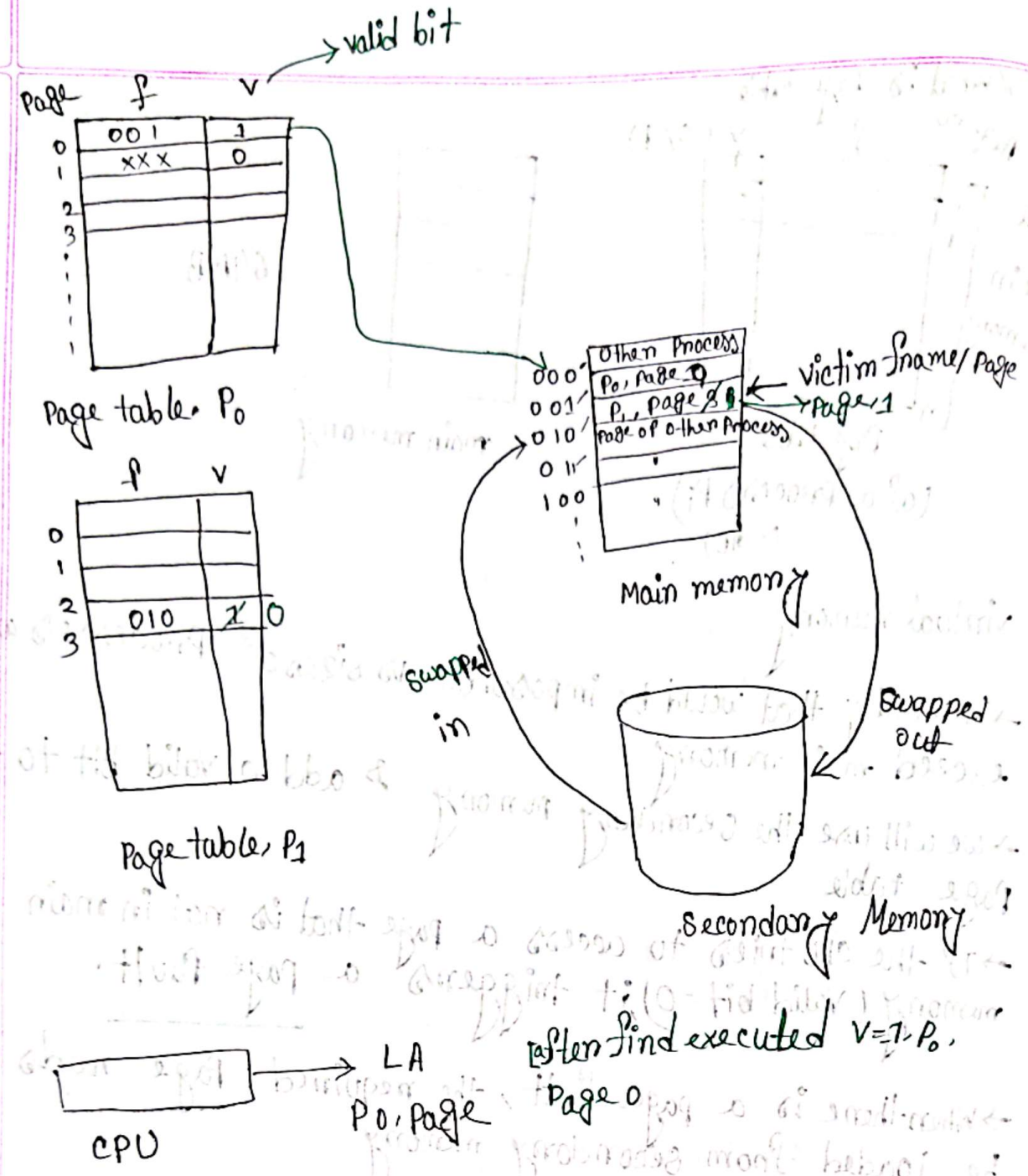
virtual memory

→ In reality that would be impossible, as sizes of processes will exceed main memory

→ we will use the secondary memory & add a valid bit to the Page table

→ If the CPU tries to access a page that is not in main memory (valid bit = 0), it triggers a page fault.

→ When there is a page fault, the required Page needs to be loaded from secondary memory



When Page 1 check V=0 not have main memory store in secondary memory we need to replace or overwrite. victim frame swapout in SM and swapin Page 1 overwrite to Page 3 and swapped out valid bit turn decrease and turn 3.010, from 1 to 0

#we need to selected victim frame.

Hit → when only page in main memory page

↑ it only happened when V=1 hit most more and more

Page Replacement Algorithms:

Hit : Requested page is in Primary memory

Miss: Requested page is:

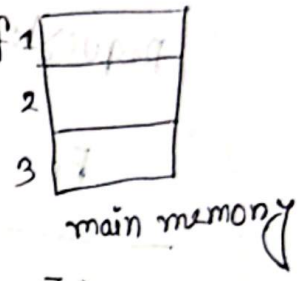
1) Not in Primary memory → space is available (frame)
2) Not in primary memory → " " unavailable (frame)

FIFO (First In, First out) Algorithm

consider 3 Frames for a Process Request → 5 different Pages

Requested : 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2

Calculate Hit & Miss Ratio



→ time

Requested	2	3	2	1	5	2	4	5	3	2	5	2
F_1	2	2	2	2	5	5	5	5	3	3	3	3
F_2		3	3	3	3	2	2	2	2	2	5	5
F_3				1	1	1	4	4	4	4	4	2
Hit/ Miss	M	M	Hit (already have 2)	M	M	M	M	H	M	H	M	M

* main memory empty Miss happend SM fetch

* tricks: Which number have most of the time that kicked first

Hit Ration = $\frac{3}{12} = 0.25$ / 25% ratio to hit

Miss Ration = $\frac{9}{12} = 0.75$ / 75% ratio to Miss

↳ time of costly to order in secondary memory

Optimal Page replacement

consider 3 Frames

For a process → 5 different pages

Requested : 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2

calculate : Hit & Miss Ratio

Requested	2	3	2	1	5	2	4	5	3	2	5	2
F_1	2	2	2	2	2	2	4	4	4	2	2	2
F_2		3	3	3	3	3	3	3	3	3	3	3
F_3					1	5	5	5	5	5	5	5
Hit/Miss	M	M	H	M	M	H	M	H	H	M	H	H

* if increase size of frame hit rate will be increase

Kicked out the one which which is latest which one have in future and can that's picked and replaced.

Hit increase but can't implemented that's

Hit ration = $\frac{6}{12} = 0.5$

Miss " = 0.5

least Recently Used (LRU)

Requested	2	3	2	1	5	2	4	5	3	2	5	2
F_1	2	2	2	2	2	2	2	2	3	3	3	3
F_2		3	3	3	5	5	5	5	5	5	5	5
F_3				1	1	1	4	4	4	2	2	2
Hit/Miss	M	M	H	M	M	H	M	H	M	M	H	H

we can't look into the futur only look for Past and least Recently

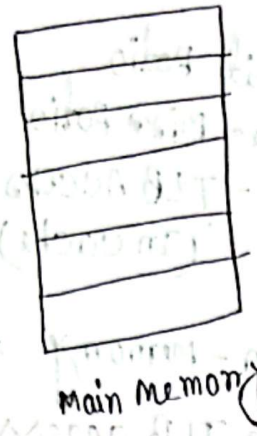
$$\text{Hit ration} = \frac{5}{12}$$

$$\text{Miss Ratio} = \frac{7}{12}$$

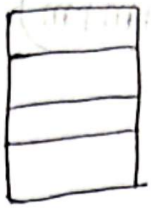
Better than FIFO but worst when optimal it's have in medal

Translation Lookaside Buffer (TLB)

28-04-26

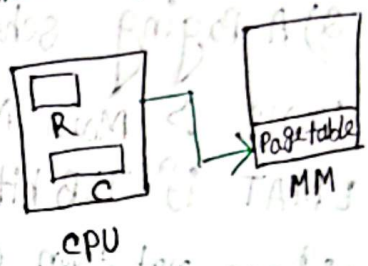


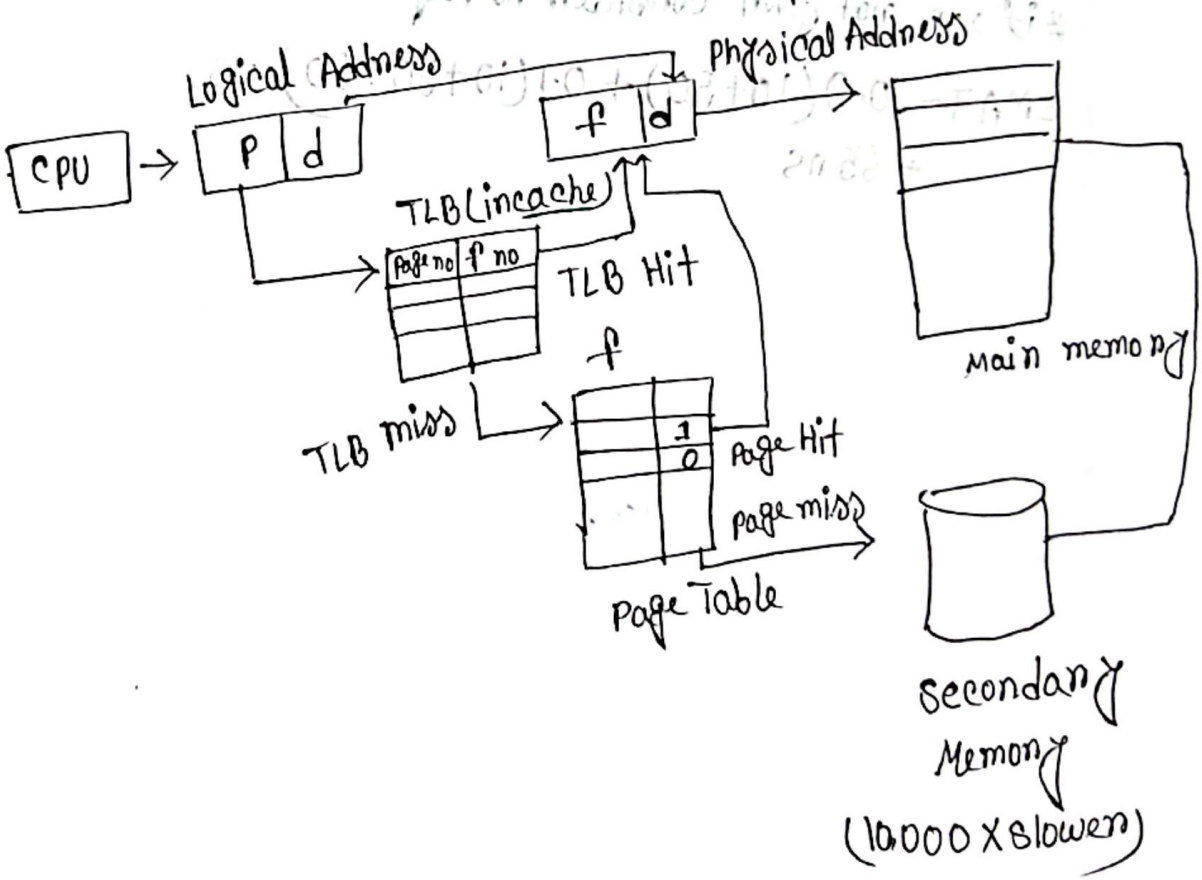
Page no f



Page table and Page index also have in main memory

when don't have cach check in Page table





Effective Memory Access Time (EMAT)

→ why reason → TLB check Page table in main m

$= h(t+m) + (1-h)(t+m+m)$

h = hit Ratio

1-h = Miss Ratio

t = TLB Access time (In cache)

m = Memory Access time

9) A Paging scheme uses a TLB, with TLB access taking 10 ns. & Main Memory Access taking 50ns what is the EMAT if TLB hit ratio is 0.9 & there are no page faults

if xm not given consider no page faults

EMAT = $0.9(10+50) + 0.1(10+50+50)$ ns

$\doteq 65$ ns